



**Department of Electronics and communication
Engineering**

Project Report

CAN Communication Protocol

Team no : 10

A Shahid Akhil	: PES2UG24EC801
Bhanuprakash.H	: PES2UG23EC033
Jayant Janrdhan Naik	: PES2UG24EC802
Ananta	: PES2UG23EC013

CAN Communication Between Multiple Nodes using MCP2515 and Arduino/ESP8266

1. Objective

This project demonstrates data communication between multiple microcontroller nodes using the CAN (Controller Area Network) protocol with MCP2515 CAN Bus modules.

Two sender nodes transmit CAN frames with different message IDs (0x100 and 0x200), while one receiver node receives and displays the data on a Serial Monitor.

This setup simulates how multiple electronic control units (ECUs) communicate in a real vehicle or industrial CAN network.

2. Advantage

- Reliable communication - strong error detection and correction.
 - Multi-master system - any node can transmit when the bus is free.
 - Efficient arbitration - high-priority messages get sent first.
 - Noise immunity - differential signaling reduces interference.
 - Reduced wiring - only two wires connect all nodes.
-

3. Applications

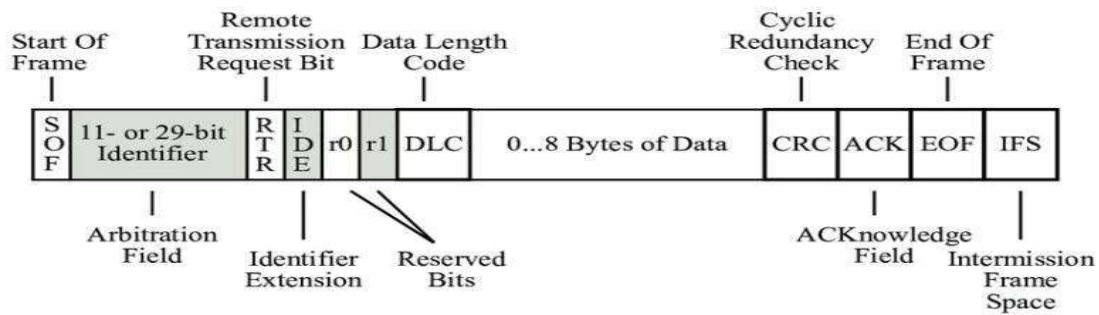
- Automotive systems (engine control, airbags, ABS, lighting)
 - Industrial automation
 - Medical equipment
 - Robotics and drones
-

4. CAN Frame Structure

A **CAN frame** is the basic unit of data transmitted over the CAN bus. It contains several fields that ensure reliable and organized communication between nodes.

Main fields of a CAN data frame:

1. **SOF (Start of Frame):**
Indicates the beginning of a new message.
2. **Arbitration Field:**
Contains the **Identifier (ID)** and **RTR bit**. The ID decides the **priority** of the message—lower ID means higher priority.
3. **Control Field:**
Specifies the **Data Length Code (DLC)**, which tells how many bytes of data follow.
4. **Data Field:**
Carries up to **8 bytes** of actual data in Standard CAN.
5. **CRC Field:**
Contains a **Cyclic Redundancy Check** value for error detection.
6. **ACK Field:**
Used to confirm successful reception of the frame by other nodes.
7. **EOF (End of Frame):**
Marks the end of the CAN message.



Problem1: Basic CAN Node-to-Node Communication

- Objective

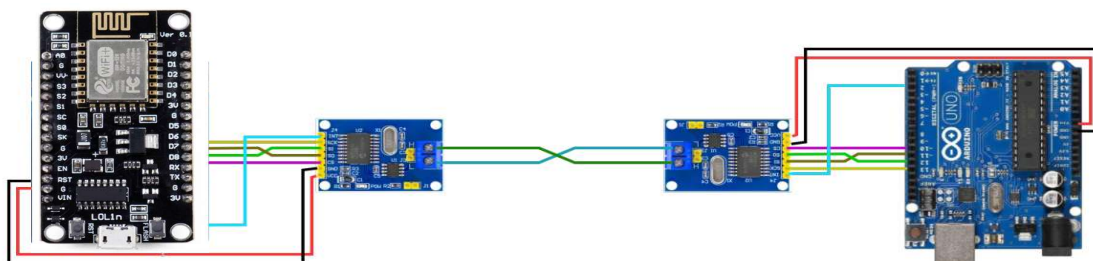
Understand the structure of a CAN frame.

Learn how two nodes exchange messages over CAN.

1. Components Required

No	Component	Quantity
1	Arduino UNO (Receiver)	1
2	ESP8266 NodeMCU (Sender 1)	1
3	MCP2515 CAN Bus Module (with TJA1050 transceiver)	3
4	Breadboard and Jumper wires	As required
5	120-ohm resistors for CAN termination (recommended)	2

2. Diagram



This setup demonstrates a two-node CAN network, where the ESP8266 sends data and the Arduino Uno receives it via the MCP2515 CAN transceivers using CANH and CANL lines.

1. ESP8266 (NodeMCU):

- Acts as the transmitter node.
- Connected to the first MCP2515 CAN module via SPI (pins: MOSI, MISO, SCK, CS).
- Provides power (3.3V or 5V) and ground to the module.

2. MCP2515 Modules:

- Two MCP2515 modules are connected together via CANH and CANL lines.
- These lines form the CAN bus, which carries differential signals for data communication.
- Usually, 120 Ω termination resistors are placed at both ends for signal stability.

3. Arduino Uno:

- Acts as the receiver node.
 - Connected to the second MCP2515 module using SPI communication.
 - Receives CAN messages sent from the ESP8266 through the CAN bus.
-

Circuit Connections

(a) Receiver Node (Arduino UNO MCP2515) MCP2515 to Arduino UNO		(b) Sender Node 1 (ESP8266 + MCP2515)	
MCP2515 Pin	Arduino Pin	MCP2515 Pin	ESP8266 Pin
VCC	5V	VCC	3.3V
GND	GND	GND	GND
CS	10	CS	D8 (GPIO15)
SO (MISO)	12	SO (MISO)	D6
SI (MOSI)	11	SI (MOSI)	D7
SCK	13	SCK	D5

Reciever1.ino

```

1  #include <SPI.h>
2  #include <mcp2515.h>
3
4
5  #define CAN_CS_PIN 10
6  #define Led1 9
7  #define Led2 8
8
9  MCP2515 mcp2515(CAN_CS_PIN);
10 struct can_frame canMsg;
11
12 void setup() {
13     Serial.begin(115200);
14     SPI.begin();
15     pinMode(Led1, OUTPUT);
16     pinMode(Led2, OUTPUT);
17
18     mcp2515.reset();
19     mcp2515.setBitrate(CAN_500KBPS, MCP_8MHZ);
20     mcp2515.setNormalMode();
21
22     Serial.println("Listening for CAN frames...\n");
23 }
24
25 void loop() {
26     if (mcp2515.readMessage(&canMsg) == MCP2515::ERROR_OK) {
27         unsigned long id = canMsg.can_id & 0x7FF; // 11-bit ID
28
29         Serial.println("===== CAN FRAME =====");
30         Serial.print("ID: 0x");
31         Serial.println(id, BIN);
32
33         Serial.print("DLC: 0x");
34         Serial.println(canMsg.can_dlc, BIN);
35
36         Serial.print("DATA: ");
37         for (uint8_t i = 0; i < canMsg.can_dlc; i++) {
38             Serial.print("0x");
39             if (canMsg.data[i] < 0x10) Serial.print("0");
40             Serial.print(canMsg.data[i], BIN);
41             Serial.print(" ");
42         }
43         Serial.println("\n=====");
44
45         // LED logic
46         if (id == 0x100) {
47             digitalWrite(Led1, HIGH);
48             delay(300);
49             digitalWrite(Led1, LOW);
50         }
51         else if (id == 0x200) {
52             digitalWrite(Led2, HIGH);
53             delay(300);
54             digitalWrite(Led2, LOW);
55         }
56
57         delay(300);
58     }
59 }
60

```

```

Sender1.ino
1  #include <SPI.h>
2  #include <mcp_can.h>
3
4  #define CAN_CS 15    // D8
5  #define LED_PIN 2    // D4 onboard LED (active LOW)
6
7  MCP_CAN CAN0(CAN_CS);
8
9  void setup() {
10     Serial.begin(9600);
11     pinMode(LED_PIN, OUTPUT);
12     digitalWrite(LED_PIN, HIGH); // LED off initially
13
14     if (CAN0.begin(MCP_ANY, CAN_500KBPS, MCP_8MHZ) == CAN_OK) {
15         Serial.println("CAN1 Init OK!");
16     } else {
17         Serial.println("CAN1 Init Failed!");
18         while (1);
19     }
20     CAN0.setMode(MCP_NORMAL);
21     Serial.println("Sender 1 Ready (ID: 0x100)");
22 }
23
24 void loop() {
25     byte data[8] = {170,170,170,170,170,170,170,170};
26
27     if (CAN0.sendMsgBuf(0x1FF, 0, 8, data) == CAN_OK) {
28         Serial.println("Message Sent from ID 0x000");
29         digitalWrite(LED_PIN, LOW); // turn LED on
30         delay(100);
31         digitalWrite(LED_PIN, HIGH); // turn LED off
32     } else {
33         Serial.println("Send Error - ID 0x000");
34     }
35     delay(1000);
36 }

```

Code : <https://github.com/Jayant077/CAN-Protocol/blob/main/Node-Node%20Communication>

Example Output on Serial Monitor:

===== CAN FRAME =====

ID: 0x100

DLC: 8

Data: 0x170 , 0x170 , 0x170 , 0x170 , 0x170 , 0x170 , 0x170 , 0x170 ,

=====

Problem2: CAN Bus Load & Arbitration Test

Objective

Observe how bus load affects message delivery.

Understand arbitration by message ID.

1. Components Required

No	Component	Quantity
1	Arduino UNO (Receiver)	1
2	ESP8266 NodeMCU (Sender 1)	1
3	ESP8266 NodeMCU (Sender 2)	1
4	Breadboard and Jumper wires	As required
5	120-ohm resistors for CAN termination (recommended)	2
6	LEDs	2
7	Ultrasonic sensor	1
8	DHT11 Humidity sensor	1

□ Bus load:

- When multiple CAN nodes send messages at the same time, the total number of messages on the bus increases – this is called the **bus load**.
- A higher bus load can cause **delays** in message delivery or **retransmissions** if two nodes try to send at once.
- By observing message timing and response, we can understand how efficiently the CAN protocol handles heavy communication traffic.

Arbitration

ECU1	0	0	1	0	0	0	1	1	0	1	0	0	CAN Data
ECU2	0	0	1	1	Loose Arbitration								
CAN-BUS	0	0	1	0	0	0	1	1	0	1	0	0	CAN Data

Start Of Frame

ECU1 will win arbitration with wired '0' And logic and final value will be '0'

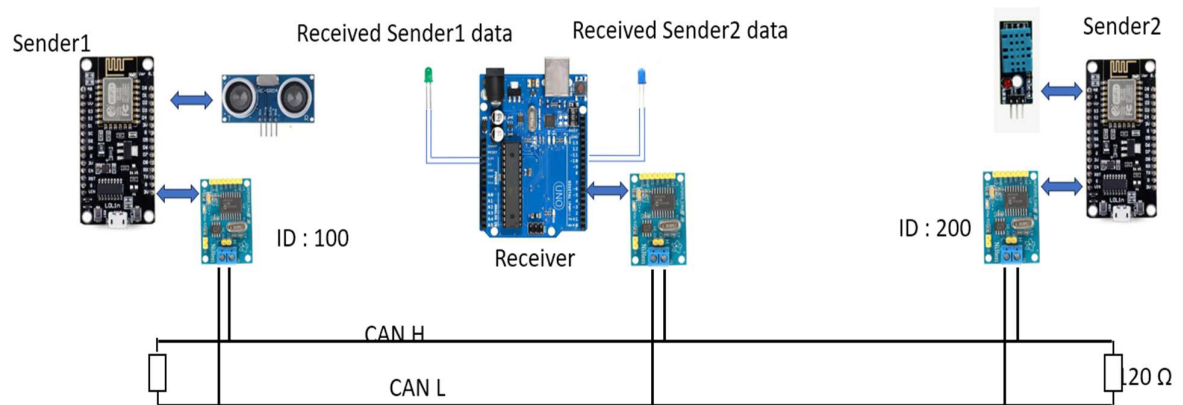
ECU2 Loose the CAN Bus and wait until ECU1 is not finished

Arbitration in CAN:

When multiple nodes transmit at the same time, CAN uses **message IDs** to decide priority. The ID with the **lowest binary value (more dominant bits '0')** wins and continues transmission, while

others stop and wait. This ensures that the **highest-priority message is always delivered first** without collisions.

Example: If Node A (ID = 01), Node B (ID = 02), Node C (ID = 03) send together, Node A wins because its ID is the smallest.



Sender Nodes (ESP8266 + Sensors + MCP2515)

Node : sender 1

ESP8266 Pin	Connected To	Function
D5 (GPIO14)	MCP2515 - SCK	SPI Clock
D6 (GPIO12)	MCP2515 - MISO	Master In Slave Out
D7 (GPIO13)	MCP2515 - MOSI	Master Out Slave In
D8 (GPIO15)	MCP2515 - CS	Chip Select
D1 (GPIO5)	Ultrasonic - Trig	Trigger Signal
D2 (GPIO4)	Ultrasonic - Echo	Echo Signal
3.3V	MCP2515 & Sensor - VCC	Power Supply
GND	MCP2515 & Sensor - GND	Ground

Node : Sender 2

D5 (GPIO14)	MCP2515 - SCK	SPI Clock
D6 (GPIO12)	MCP2515 - MISO	Master In Slave Out
D7 (GPIO13)	MCP2515 - MOSI	Master Out Slave In
D8 (GPIO15)	MCP2515 - CS	Chip Select
D4 (GPIO2)	DHT11/DHT22 - DATA	Sensor Data Output
3.3V	MCP2515 & DHT - VCC	Power Supply
GND	MCP2515 & DHT - GND	Ground

Receiver Node (Arduino UNO with MCP2515 and Two LEDs)

Arduino UNO Pin	Connected To	Function
13 (SCK)	MCP2515 - SCK	SPI Clock
12 (MISO)	MCP2515 - MISO	Master In Slave Out
11 (MOSI)	MCP2515 - MOSI	Master Out Slave In
10 (CS)	MCP2515 - CS	Chip Select
5V	MCP2515 - VCC	Power Supply
GND	MCP2515 - GND	Ground
D7	LED1 (Ultrasonic Data Indicator)	Turns ON when Ultrasonic data received
D8	LED2 (DHT Data Indicator)	Turns ON when DHT data received
5V	LEDs - VCC	Power Supply for LEDs
GND	LEDs - GND	Common Ground

Code : <https://github.com/Jayant077/CAN-Protocol/blob/main/Sensor%20Data%20Exchange>

Expected Serial Monitor Output

Listening for CAN frames...

Distance: 15.2 cm

=====

Humidity: 48.7 %, Temperature: 27.5 °C

=====

Distance: 8.5 cm

=====

Humidity: 49.1 %, Temperature: 27.8 °C

=====

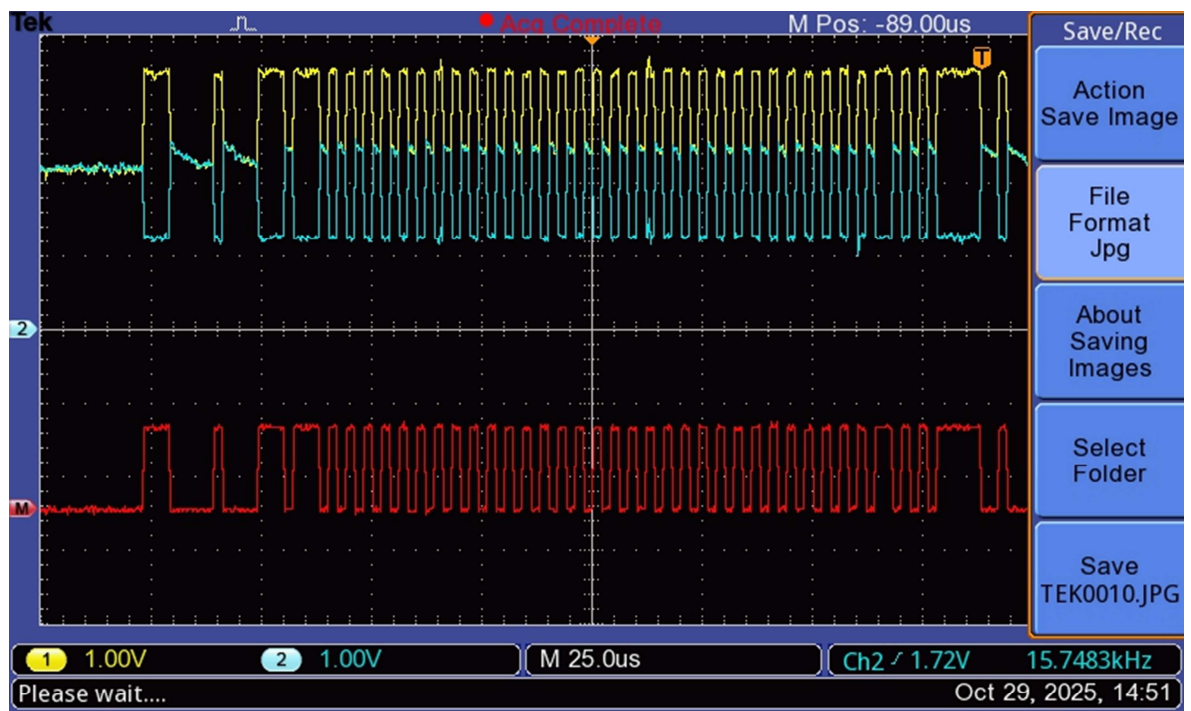
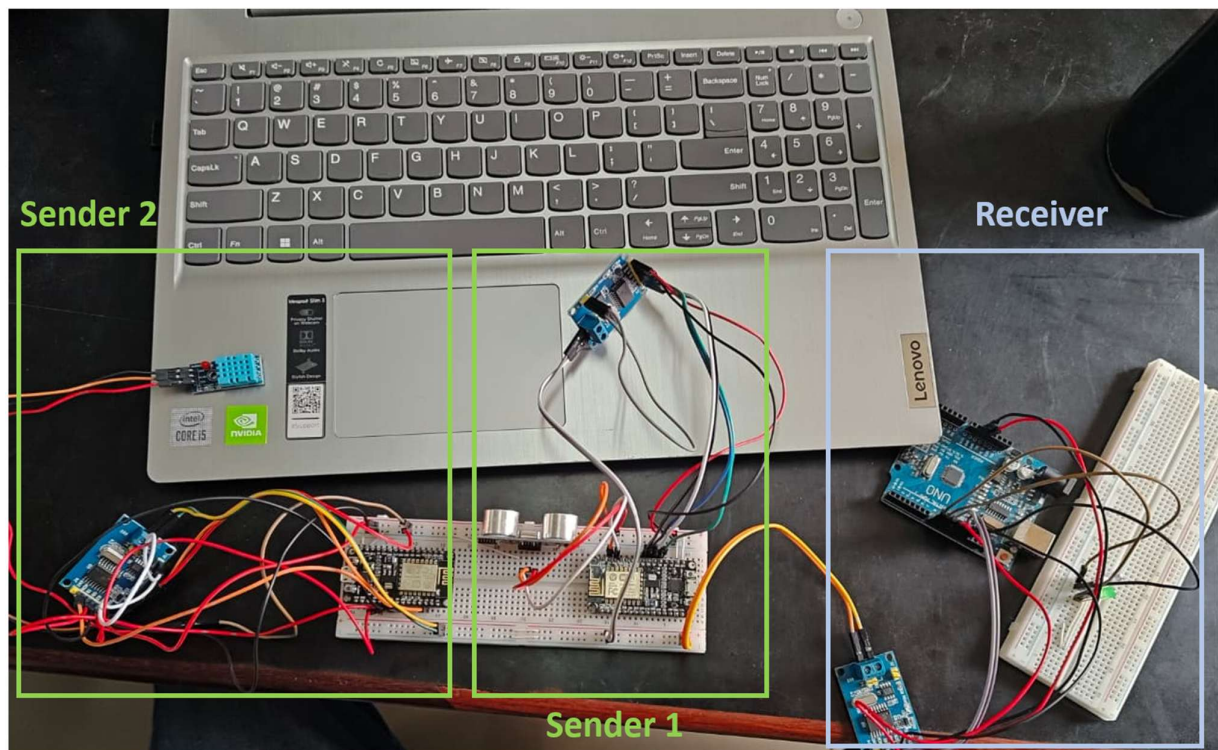
Distance: 24.3 cm

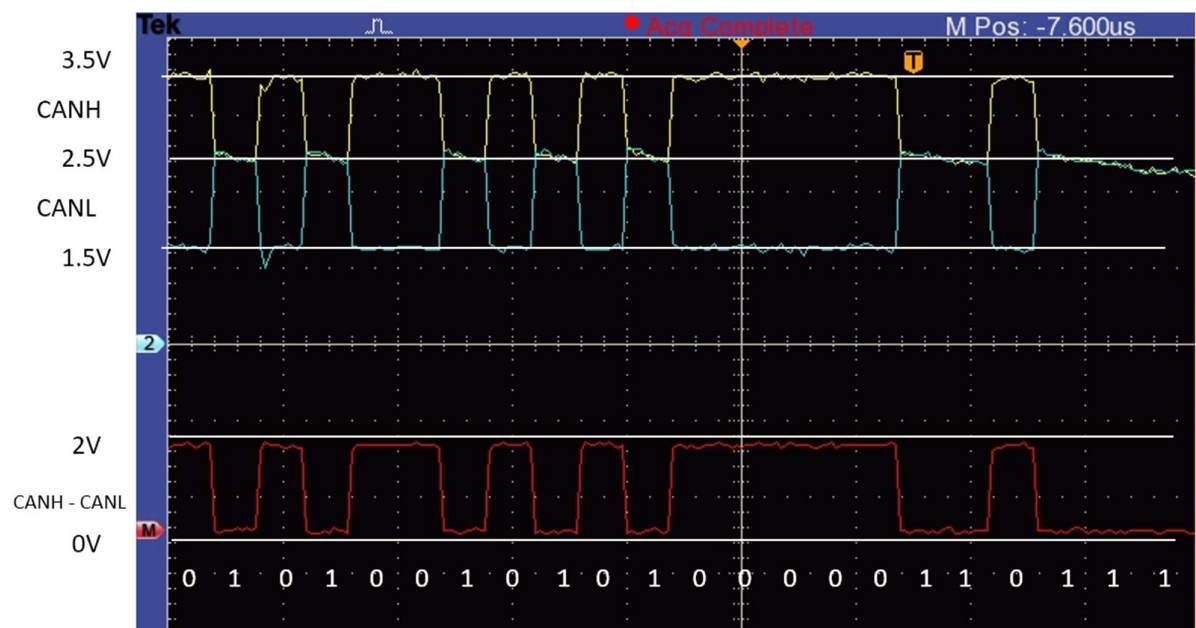
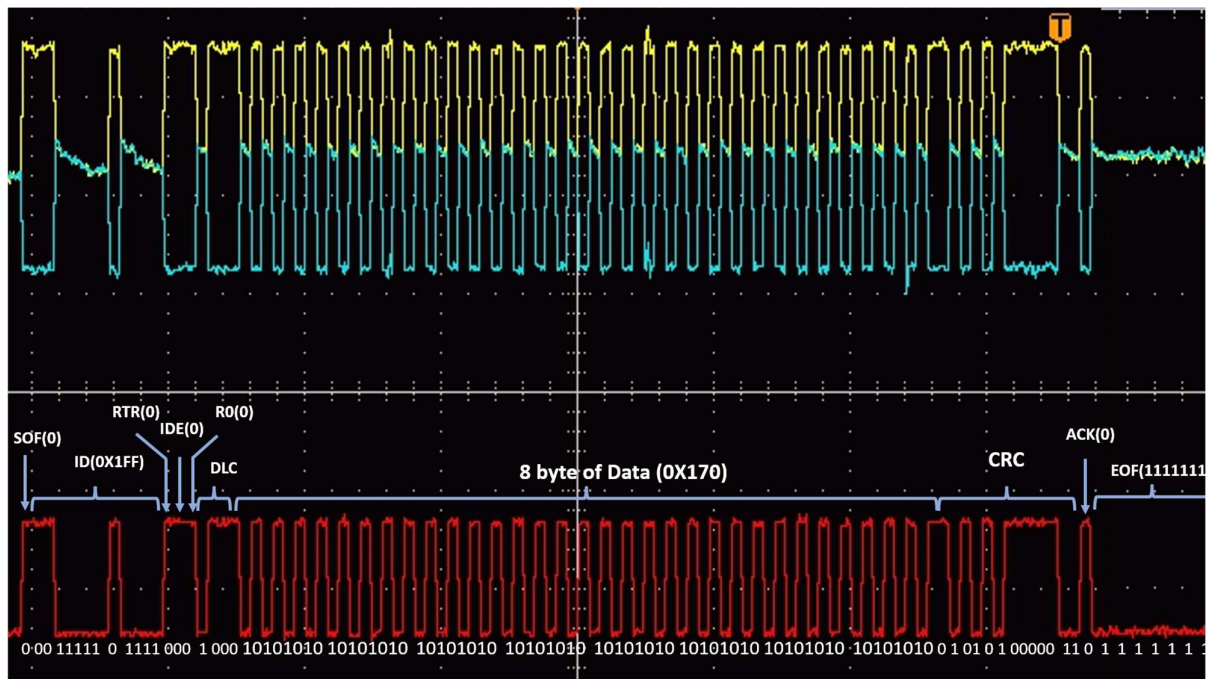
=====

Humidity: 50.0 %, Temperature: 28.2 °C

=====

Hardware Setup





Working Principle

1. Both sender nodes are connected to the same CAN bus (CAN_H and CAN_L lines).
 2. Each sender transmits its data frame periodically using a unique identifier.
 - Sender 1 → ID 0x100
 - Sender 2 → ID 0x200
 3. The receiver continuously listens on the bus.
 4. When a message is received, the MCP2515 sends it to the Arduino via SPI.
 5. The Arduino decodes the message and displays the details on Serial Monitor.
 6. Multiple senders can transmit data simultaneously — the CAN protocol handles bus arbitration automatically.
-

Key Parameters

Parameter	Value
CAN Btrrate	500 kbps
Crystal Frequency (MCP2515)	8 MHz
Message ID 1	0x100
Message ID 2	0x200
Frame Type	Standard (11-bit)
Data Length	Up to 8 bytes

Observations

- Receiver successfully identifies and displays both CAN message IDs and data bytes.
- Arbitration occurs automatically when multiple senders transmit.
- Data is displayed clearly on the Serial Monitor.
- The onboard LEDs on sender boards blink to indicate successful transmission.
- On receiver side LED's will indicate the received message ID.